Reporte de seguridad — MediCentro

Oct 14, 2026 · @Fernando Guerra

Resumen

Lo más grave no es la pantalla sin login sino la API: cualquier usuario logueado, aunque sea de Farmacia, puede cambiarle la contraseña al administrador o darse el rol ADMIN.

Se revisó el front `sistema_medicentro` (rama `Cambios-unificados`) y el backend `MediCentroBackendV3` (rama `main`). Revisión de código, sin pruebas de penetración.

Gravedad	Hallazgo
Crítica	Cualquier usuario autenticado puede cambiar la contraseña de otro usuario, sin la contraseña actual, y asignarse roles
Crítica	La llave que firma los tokens JWT está en el repositorio
Alta	Las 93 pantallas de `/home/...` abren sin login, solo con la URL
Alta	El backend no valida roles ni permisos: todo usuario logueado puede usar todos los endpoints
Media	El "código" del login es el `id_usuario` (1, 2, 3…), así que no agrega seguridad
Media	Contraseña mínima de 4 caracteres en el login; el token se guarda en `localStorage` y el logout no lo invalida
Baja	Endpoint de plantilla `/WeatherForecast` público y CORS fijo en el código

Pantallas accesibles solo con la URL

Todas las rutas internas abren sin sesión: el padre `home` tiene `canActivate: []` (vacío) en `src/app/app.routes.ts` línea 86, y el único guard, `permisosGuard`, deja pasar cuando no hay permisos cargados.

Qué	Dónde	Por qué deja entrar
Ruta `home` y sus 93 hijas (líneas 90–182)	`src/app/app.routes.ts` línea 86	`canActivate: []`: no hay guard de autenticación
`permisosGuard` (`canActivateChild`, línea 87)	`src/app/service/permisos.guard.ts` líneas 17–20	Si `modulosPermitidos()` es `null`, devuelve `true`. Sin login siempre es `null`
Cálculo de permisos	`src/app/service/permisos-menu.service.ts` líneas 92–114	Devuelve `null` si el usuario no tiene permisos o si todavía no cargaron; un usuario sin permisos configurados también ve todo

Ejemplos que abren escribiendo la URL sin haber iniciado sesión:

- `http://localhost:4200/home/mantenimiento/usuarios` (usuarios, roles, permisos)
- `http://localhost:4200/home/pacientes` y `/home/expedientes` (datos clínicos)
- `http://localhost:4200/home/facturacion` y `/home/mantenimiento/bitacora`

Qué se ve: la pantalla, el menú y los formularios. Los datos no cargan, porque la API responde 401 sin token, así que hoy la fuga real es de estructura y no de datos. El riesgo sube si algún endpoint queda sin `[Authorize]` o si la pantalla muestra datos de ejemplo.

API: autenticación y roles

La API sí exige token: 35 de 36 controladores tienen `[Authorize]`. El problema es que no distingue quién es el usuario. Ningún endpoint usa `[Authorize(Roles = ...)]` ni revisa permisos, así que el rol solo viaja en el token y nadie lo valida.

Gravedad	Endpoint	Archivo	Problema
Crítica	`PUT /api/Usuarios/{id}/contrasena`	`Controllers/UsuariosController.cs` línea 150; `Servicios/UsuariosService.cs` líneas 20–31	Cambia la contraseña de cualquier `id` sin pedir la actual ni verificar que sea el mismo usuario. Un usuario de Caja puede tomar la cuenta `admin`
Crítica	`POST /api/Usuarios/{id}/roles`	`UsuariosController.cs` línea 99	Cualquier usuario puede asignarse el rol ADMIN o cualquier otro

Gravedad	Endpoint	Archivo	Problema
Alta	`POST/PUT/DELETE /api/Usuarios`, `/api/Roles`, `/api/Permisos`	`UsuariosController.cs`, `RolesController.cs`, `PermisosController.cs`	Crear, borrar o modificar usuarios, roles y permisos solo exige estar logueado
Alta	Resto de módulos (expedientes, facturación, bitácora, eliminados…)	Todos los controladores	Un usuario de Farmacia puede leer expedientes clínicos o anular facturas por API, aunque el menú se lo oculte
Baja	`GET /WeatherForecast`	`Controllers/WeatherForecastController.cs`	Plantilla de Visual Studio sin `[Authorize]`; no expone datos, pero sobra

`EliminadosController.cs` línea 10 ya trae el comentario de agregar `[Authorize(Roles = "ADMIN")]`; falta aplicarlo ahí y en el resto.

Sesión y token

El manejo del token es correcto en lo básico (expira, el backend valida firma, emisor, audiencia y vigencia), pero el login es más débil de lo que parece.

Gravedad	Qué	Dónde	Detalle
Media	El "código" es el `id_usuario`	`Servicios/AuthService.cs` (busca por `NombreUsuario` + `IdUsuario`)	Son números consecutivos, fáciles de adivinar; en la práctica el login es usuario + contraseña
Media	Contraseña mínima de 4 caracteres	`Dto/Auth/LoginRequestDto.cs`	Al cambiar contraseña se piden 6 (`CambiarContrasenaDto.cs`); no hay regla de complejidad
Media	Token en `localStorage`	`src/app/login/login.ts` líneas 87–88	Cualquier script inyectado (XSS) puede leerlo
Baja	Logout no invalida el token	`src/app/service/auth.service.ts` líneas 37–42	Solo lo borra del navegador; un token copiado sigue sirviendo hasta que vence (60 min, `appsettings.json` línea 9)
OK	Bloqueo por intentos	`AuthService.cs`, `MaxIntentosFallidos = 5`	Bloquea tras 5 intentos fallidos; el desbloqueo es manual
OK	Mensaje genérico de error	`AuthService.cs`	No revela si el usuario existe
OK	Sesión vencida	`src/app/interceptors/auth.interceptor.ts`	Con un 401 borra el token y manda al login

Secretos y configuración

La llave del JWT está en el repositorio desde el primer commit, así que quien tenga acceso al repo puede fabricar un token válido de cualquier usuario sin conocer contraseñas.

Gravedad	Qué	Dónde	Detalle
Crítica	Llave de firma del JWT	`MediCentroBackendV3/appsettings.json` línea 6	Versionada (commit `c79ed49`); aunque se borre, queda en el historial y hay que cambiarla
Media	Cadena de conexión	`appsettings.json` línea 3	Cada quien la edita en Mac o Linux para poner usuario y contraseña de su SQL; fácil subir una contraseña por accidente
Baja	CORS fijo	`Program.cs` línea 16	Solo `http://localhost:4200`; correcto para desarrollo, pero va en el código y no en configuración
Baja	Swagger	`Program.cs` líneas 107–110	Solo en Development: correcto; confirmar que producción no corra como Development

No se encontraron `console.log` que impriman token ni contraseña en esta rama; la versión vieja del login en `main` sí imprimía la contraseña en la consola.

Recomendaciones por prioridad

1. **Cambio de contraseña (crítico).** En `PUT /api/Usuarios/{id}/contrasena`, permitirlo solo si el `id` es el del token (`ClaimTypes.NameIdentifier`) y pidiendo la contraseña actual, o si quien lo pide tiene rol ADMIN.
2. **Roles en la API (crítico).** Poner `[Authorize(Roles = "ADMIN")]` en Usuarios, Roles, Permisos, Eliminados y Bitácora. Después, una política por módulo que use la tabla `rol_permiso`.
3. **Llave del JWT (crítico).** Moverla a User Secrets en desarrollo y a una variable de entorno en producción; generar una llave nueva, porque la actual ya es pública para el equipo.
4. **Guard de autenticación en el front (alto).** Crear un `authGuard` que revise que exista el token y no haya vencido, y ponerlo en `canActivate` de la ruta `home` (`app.routes.ts` línea 86). Si no hay sesión, redirigir a `/`.
5. **`permisosGuard` (alto).** Cuando `modulosPermitidos()` es `null` porque no hay sesión, bloquear en vez de dejar pasar. Esperar a que carguen los permisos antes de decidir.
6. **Login (medio).** Que el "código" no sea el `id_usuario` (o quitarlo y depender de una contraseña fuerte), subir el mínimo a 8 caracteres y unificarlo con el de cambio de contraseña.

7. **Limpieza (bajo).** Borrar `WeatherForecastController.cs` y `WeatherForecast.cs`; mover CORS y la cadena de conexión a configuración por ambiente.

Ojo: el guard del front (puntos 4 y 5) evita que alguien vea las pantallas, pero no protege los datos. La protección real son los puntos 1 a 3 en la API, porque cualquiera puede llamarla desde Postman sin pasar por el front.